

# Python Practice Questions: Special (Dunder) Methods

---

## 1. `__eq__` (Equal)

Create a class Student with attributes name and marks.

- Override `__eq__` so that two Student objects are considered equal if their **marks** are the same.
  - Create two objects and compare them using `==`.
- 

## 2. `__lt__` (Less Than)

Create a class Product with attributes name and price.

- Override `__lt__` to compare products based on their **price**.
  - Test the comparison using `<`.
- 

## 3. `__gt__` (Greater Than)

Create a class Employee with attributes name and salary.

- Override `__gt__` to compare employees based on their **salary**.
  - Test the comparison using `>`.
- 

## 4. `__iadd__` (In-place Addition)

Create a class BankAccount with an attribute balance.

- Override `__iadd__` so that `account += amount` adds money to the balance.
  - Display the updated balance after the operation.
- 

## 5. `__isub__` (In-place Subtraction)

Create a class Wallet with an attribute money.

- Override `__isub__` so that `wallet -= amount` deducts money from the wallet.
  - Print the remaining amount after the operation.
-

## 6. `__len__`

Create a class Library.

- Store a list of books.
  - Override `__len__` so that `len(library)` returns the total number of books.
- 

## 7. `__getitem__`

Create a class Playlist.

- Store a list of songs.
  - Override `__getitem__` so that songs can be accessed using indexing (e.g., `playlist[0]`).
- 

## 8. `__setitem__`

Create a class Marksheet.

- Store marks in a list.
  - Override `__setitem__` so that marks can be updated using indexing (e.g., `marksheet[1] = 95`).
- 

## 9. `__contains__`

Create a class Classroom.

- Store a list of student names.
  - Override `__contains__` so that `"Rahul"` in classroom checks whether Rahul exists in the classroom.
- 

## 10. `__iter__`

Create a class Team.

- Store a list of players.
  - Override `__iter__` so that a for loop can iterate through all players.
- 

## 11. `__bool__`

Create a class Cart.

- Store items in a shopping cart.
  - Override `__bool__` so that the cart is considered True when it contains items and False when it is empty.
- 

## 12. `__call__`

Create a class Calculator.

- Store a number in the object.
  - Override `__call__` so that calling the object like a function multiplies the stored number by a given value.
- 

## Mixed Practice Questions

### Q13. MovieCollection

Create a class MovieCollection that supports:

- `len(obj)`
- `obj[0]`
- "Avatar" in obj

using `__len__`, `__getitem__`, and `__contains__`.

---

### Q14. CricketTeam

Create a class CricketTeam that supports:

- Iteration using a for loop
- Length using `len()`
- Membership testing using `in`

using `__iter__`, `__len__`, and `__contains__`.

---

### Q15. ScoreBoard

Create a class ScoreBoard that supports:

- `score1 == score2`
- `score1 > score2`

- `score1 < score2`

using `__eq__`, `__gt__`, and `__lt__`.

---

### **Q16. Inventory (Interview Level)**

Create a class `Inventory`.

- Allow item access using indexing (`__getitem__`).
  - Allow item modification using indexing (`__setitem__`).
  - Return the total number of items (`__len__`).
  - Support iteration through all items (`__iter__`).
- 

### **Q17. DigitalWallet (Advanced)**

Create a class `DigitalWallet`.

- Support `+=` for adding money (`__iadd__`).
- Support `-=` for spending money (`__isub__`).
- Return `False` when the balance is 0 and `True` otherwise (`__bool__`).
- Compare two wallets using `>` (`__gt__`).